

Advanced Database Management Systems

CST 327–2

Disk Storage, File Structures and Hashing

(Part 1 – Disk Storage)

Storage Devices

- Data must be stored physically on some computer **storage medium**.
- The DBMS software can then retrieve, update, and process this data as needed.

Categories of Storage Hierarchy

- Primary storage
 - can be operated on directly by the computer's CPU
 - E.g.:
Main memory, cache memory
- Secondary and tertiary storage
 - cannot be processed directly by the CPU; first it must be copied into primary storage and then processed by the CPU
 - E.g.:
Secondary – hard disk
Tertiary - optical disk, magnetic tape

Storage of Databases

- Databases typically store large amounts of data that must persist over long periods of time.
- Parts of this data are accessed and processed repeatedly during this period.
- Most databases are stored permanently on magnetic disk secondary storage. Why???

Storage of Databases cont.

- Reasons:
 - Normally, databases are too large to fit entirely in main memory.
 - When databases are stored on secondary devices, it can minimize permanent loss of stored data as secondary storage devices are nonvolatile.
 - The cost of storage per unit of data in secondary storage is less than primary storage.
 - Secondary and tertiary storage devices usually have a larger capacity than primary storage.

Storage of Databases cont.

- The data stored on disk is organized as files of records.
- Each record is a collection of data values that can be interpreted as facts about entities, their attributes, and their relationships.
- Records should be stored on disk that makes it possible to locate them efficiently when they are needed.

Primary File Organizations

- It determines how the file records are physically placed on the disk, and how the records can be accessed.
- Several types:
 - Heap file (unordered file)
 - places the records on disk in no particular order by appending new records at the end of the file
 - Sorted file (sequential file)
 - keeps the records ordered by the value of a particular field (called the sort key)

Primary File Organizations cont.

- Hashed file
 - uses a hash function applied to a particular field (called the hash key) to determine a record's placement on disk
- B-trees
 - use tree structures

Secondary Storage Devices

- Magnetic disk
- Magnetic tape



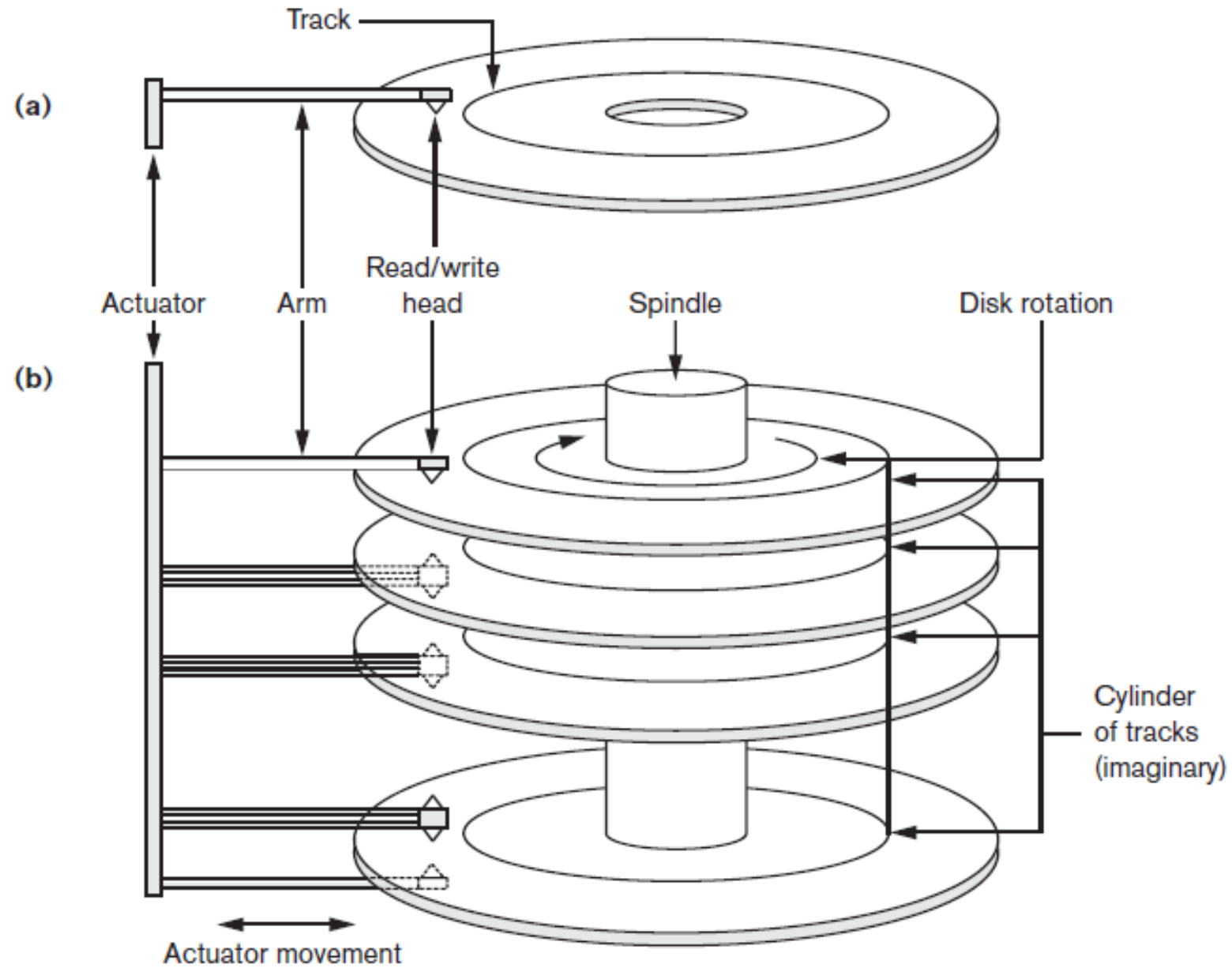
Magnetic Disk Storage



- Magnetic disks are used for storing large amounts of data.
- The most basic unit of data on the disk is a single **bit** of information.
- By magnetizing an area on disk in certain ways, one can make it represent a bit value of either 0 (zero) or 1 (one).
- To code information, bits are grouped into **bytes** (or **characters**).

Magnetic Disk Storage cont.

- (a) A single-sided disk with read/write hardware.
(b) A disk pack with read/write hardware.



Magnetic Disk Storage cont.

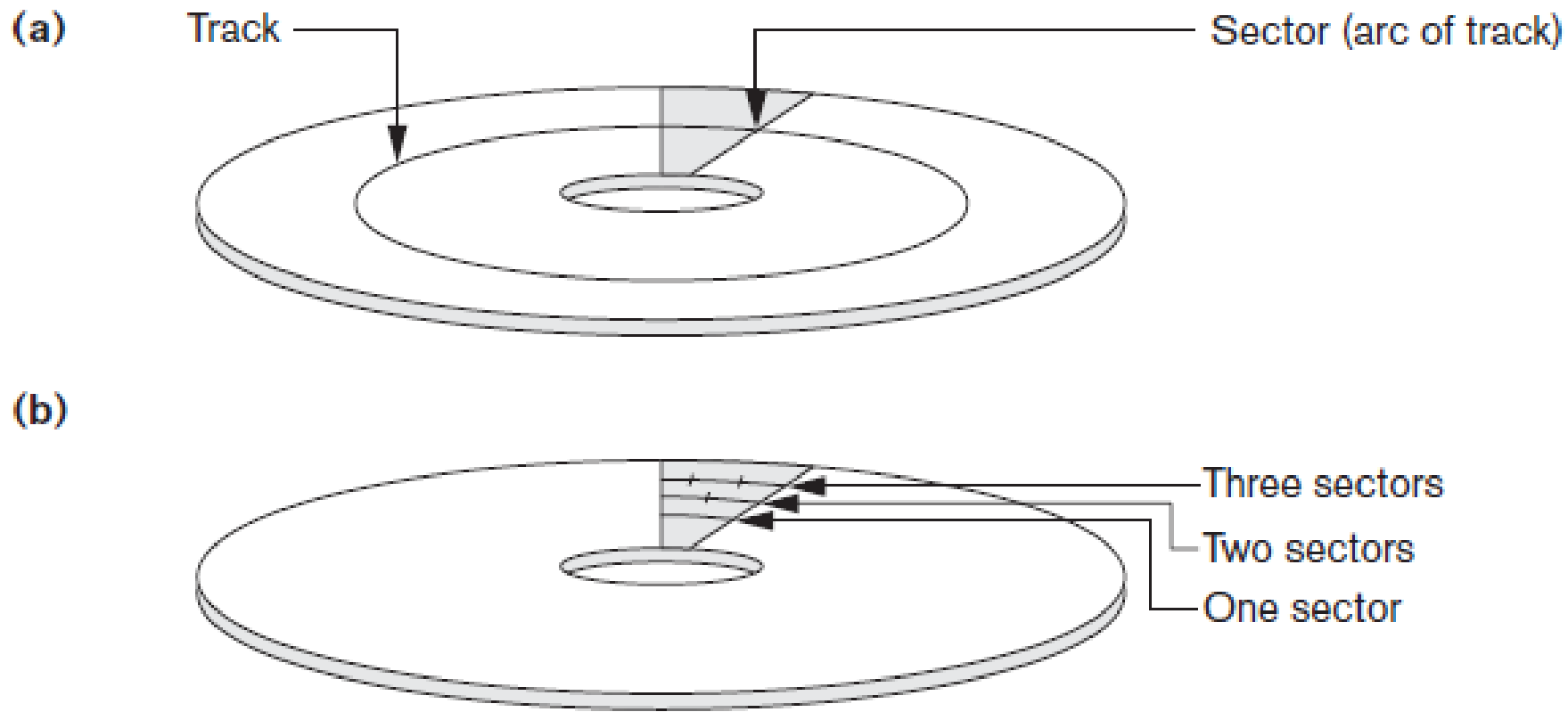


- A disk is **single-sided** if it stores information on one of its surfaces only and **double-sided** if both surfaces are used.
- To increase storage capacity, disks are assembled into a **disk pack**, which may include many disks and therefore many surfaces.
- Information is stored on a disk surface in concentric circles, each having a distinct diameter and each circle is called a **track**.

Magnetic Disk Storage cont.

- In disk packs, tracks with the same diameter on the various surfaces are called a **cylinder**.
- Since, a track usually contains a large amount of information, it is divided into smaller blocks or sectors.
- The division of a track into **sectors** is hard-coded on the disk surface and cannot be changed. – A disk can have different sector organizations.

Magnetic Disk Storage cont.



Different sector organizations on disk. (a) Sectors subtending a fixed angle. (b) Sectors maintaining a uniform recording density.

Magnetic Disk Storage cont.

- A disk is a random access addressable device.
- Transfer of data between main memory and disk takes place in units of disk blocks.
- The **hardware address** of a block is the combination of a cylinder number, track number (surface number within the cylinder on which the track is located), and block number (within the track).

Seek time, Rotational Delay (Latency) and Block Transfer Time

- To transfer a disk block, to its address, the disk controller must first mechanically position the read/write head on the correct track. The time required to do this is called the **seek time**.
- Following that, next delay is the **rotational delay** or **latency**, while the beginning of the desired block rotates into position under the read/write head. It depends on the rpm of the disk.
- Finally, some additional time is needed to transfer the data, called the **block transfer time**.

Magnetic Tape Storage Devices

- Magnetic tapes are sequential access devices; to access the n^{th} block on tape, first we must scan the preceding $n - 1$ blocks.
- Data is stored on reels of high-capacity magnetic tape, somewhat similar to audiotapes or videotapes.
- A tape drive is required to read the data from or write the data to a **tape reel**.
- Usually, each group of bits that forms a byte is stored across the tape, and the bytes themselves are stored consecutively on the tape; therefore, we access the data blocks in **sequential order**.

Activity 01

- Assume that a disk pack contains 50 surfaces. Each disk surface contains 5,000 tracks, with 100 sectors in a track. The sector size is 512 Bytes, and each block has 40 sectors. The track platters are double-sided and rotate at 7,200 rpm (revolutions per minute).
 - i. How many disk platters and cylinders are there in the disk pack?
 - ii. What is the capacity of the disk pack?
 - iii. Compute the data transfer rate if one data track can be transferred per revolution.

Activity 01 – Answer

- i. How many disk platters and cylinders are there in the disk pack?
 - No. of disk platters = $50 / 2 = 25$
 - No. of cylinders = 5000
- ii. What is the capacity of the disk pack?
 - Capacity of surface = $512 * 100 * 5000$ bytes
 - Capacity of the disk pack = $512 * 100 * 5000 * 50$ bytes

Activity 01 – Answer

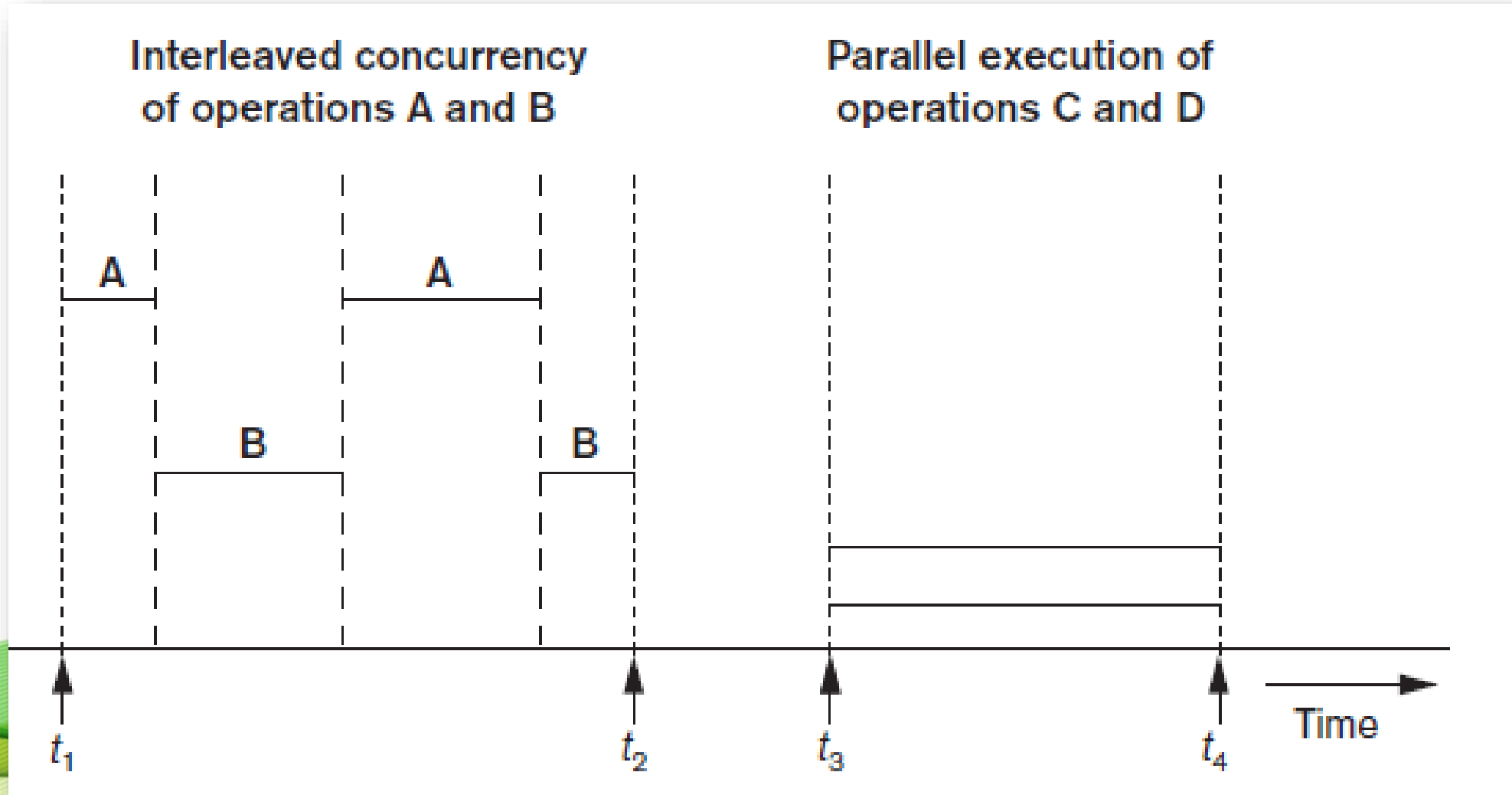
iii. Compute the data transfer rate if one data track can be transferred per revolution.

- Capacity of a track = $512 * 100$ bytes
- Data transfer rate = $512 * 100 \text{ bytes} * 7200/60 \text{ seconds}$
= 6 144 000 bytes per second

Buffering of Blocks

- When several blocks need to be transferred from disk to main memory and all the block addresses are known, several buffers can be reserved in main memory to speed up the transfer.
- While one buffer is being read or written, the CPU can process data in the other buffer because an independent disk I/O processor (controller) exists that, once started, can proceed to transfer a data block between memory and disk independent of and in parallel to CPU processing.

Interleaved Concurrency vs. Parallel Execution



Activity 02

- What are the differences between interleaved concurrency and parallel execution?

Placing File Records on Disk

Records and Record Types

- Data is usually stored in the form of records.
- Each record consists of a collection of related data values or items, where each value is formed of one or more bytes and corresponds to a particular field of the record.
- Records usually describe entities and their attributes.
- E.g.: Employee entity has attributes such as Name, DOB, Salary, etc.

Placing File Records on Disk cont.

- A collection of field names and their corresponding data types constitutes a record type or record format definition.
- A data type, associated with each field, specifies the types of values a field can take.

Placing File Records on Disk cont.

- E.g.: EMPLOYEE record type defined using the C programming language

```
struct employee{  
    char name[30];  
    char ssn[9];  
    int salary;  
    int job_code;  
    char department[20];  
} ;
```

Activity 03

- Consider a disk with following characteristics.
 - Sector size is 512 bytes; each block contains 15 sectors, and the number of sectors per track is 450; number of tracks per surface is 400. The disk pack contains 20 double sided disks.
- a) How many cylinders are there in the disk pack?
- b) What is the capacity of a track and a cylinder?
- c) Compute the capacity of the disk pack.
- d) Assuming that the disk platters rotate at a speed of 2400 rpm, and one track can be transferred per revolution, find the transfer rate.

Files, Fixed-Length Records and Variable-Length Records

- A **file** is a sequence of records.
- In many cases, all records in a file are of the same record type.
- Some files are having **fixed-length records** and some are having **variable-length records**.

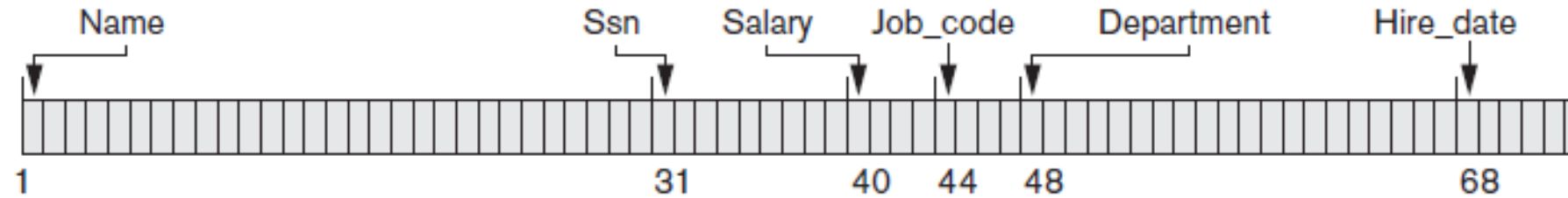
Files, Fixed-Length Records and Variable-Length Records cont.

- Fixed-length records
 - If every record in the file has exactly the same size (in bytes), the file is said to be made up of **fixed-length records**.
- Variable length records
 - If different records in the file have different sizes, the file is said to be made up of **variable-length records**.

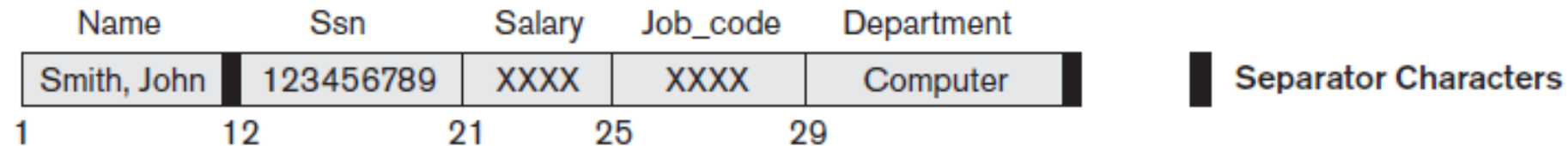
Why files may have variable-length records?

- The file records are of the same record type, but one or more of the fields are of varying size (variable-length fields). E.g.: Names of Employees
- The file records are of the same record type, but one or more of the fields may have multiple values for individual records (repeating fields).
- The file records are of the same record type, but one or more of the fields are optional (optional fields).
- The file contains records of different record types and hence of varying size (mixed file).

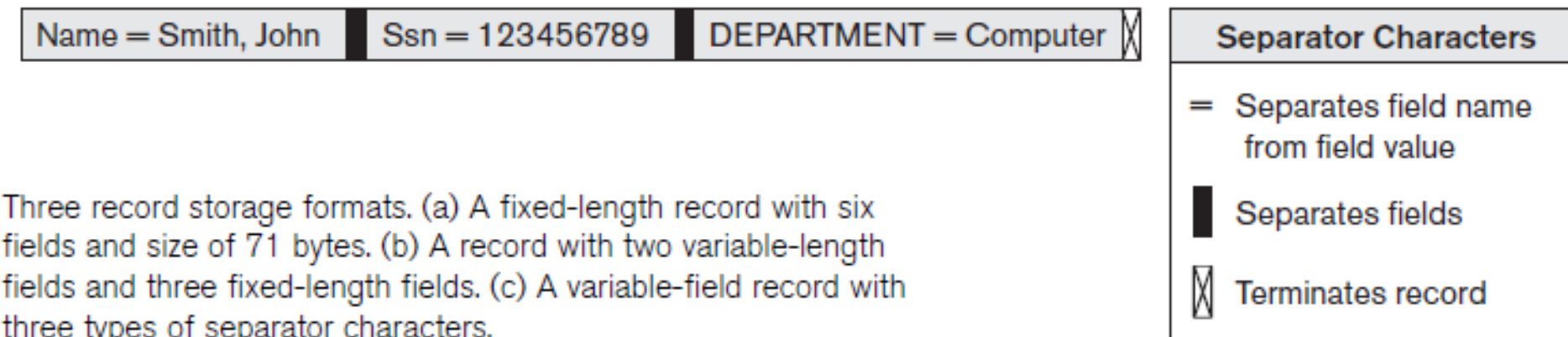
(a)



(b)



(c)



Three record storage formats. (a) A fixed-length record with six fields and size of 71 bytes. (b) A record with two variable-length fields and three fixed-length fields. (c) A variable-field record with three types of separator characters.

Record Blocking

- A block is the unit of data transfer between disk and memory.
- Therefore, the records of a file must be allocated to disk blocks - Blocking.
- When the block size is larger than the record size, it can contain many number of records and sometimes a record may not fit to only one block.

Record Blocking cont.

- If block size is **B** bytes, for a file of fixed-length records of size **R** bytes, with **$B \geq R$** ;

$$\text{bfr} = B/R \text{ records per block}$$

(rounds down the number to an integer)

- The value bfr is called the **blocking factor** for the file.
- In general, R may not divide B exactly, so we have some unused space in each block.

$$\text{Unused space} = B - (\text{bfr} * R) \text{ bytes}$$

Activity 04

- Assume that the block size is 100 bytes and there are 6 records with the record size is 15 bytes. Calculate the blocking factor and the unused space on the disk once all the records are allocated on the disk.

Spanned Records vs. Unspanned Records

- To utilize this unused space, part of a record can be stored on one block and the rest on another.
- A pointer at the end of the first block points to the block containing the remainder of the record if it is not the next consecutive block on disk.
- This organization is called **spanned** because records can span more than one block.

Spanned Records vs. Unspanned Records cont.

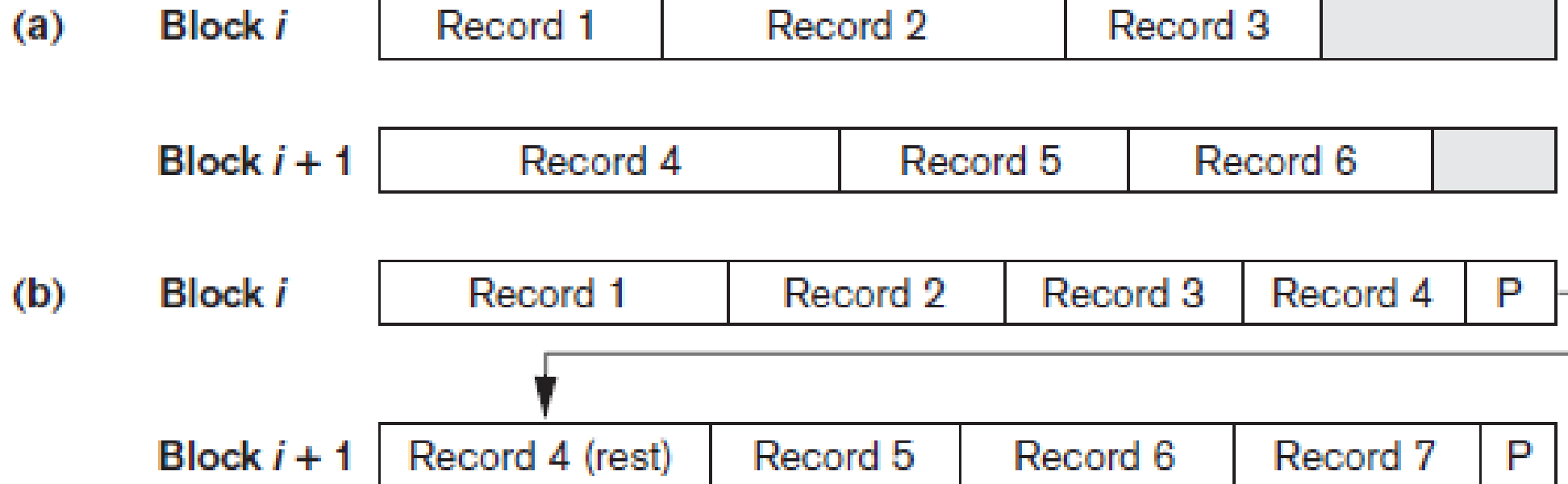
- If records are not allowed to cross block boundaries, the organization is called **unspanned**.
- This is used with fixed-length records having $B > R$ because it makes each record start at a known location in the block. This simplifies record processing.
- For variable-length records, either a spanned or an unspanned organization can be used.
- If the average record is large, it is advantageous to use spanning to reduce the lost space in each block.

Spanned Records vs. Unspanned Records cont.

Types of record organization.

(a) Unspanned.

(b) Spanned.



Spanned Records vs. Unspanned Records cont.

- For variable-length records using spanned organization, each block may store a different number of records.
- The blocking factor (**bfr**) represents the average number of records per block for the file.
- The number of blocks **b** needed for a file of **r** records;
$$b = (r/bfr) \text{ blocks}$$

(rounds the value up to the next integer)

Allocating File Blocks on Disk

- Several techniques:
 - Contiguous Allocation
 - Linked Allocation
 - Indexed Allocation

Allocating File Blocks on Disk cont.

Contiguous Allocation

- file blocks are allocated to consecutive disk blocks
- able to read the whole file very fast - using double buffering
- expanding the file is difficult

Linked Allocation

- each file block contains a pointer to the next file block
- easy to expand the file
- slow to read the whole file

Allocating File Blocks on Disk cont.

- A combination of the two, allocates **clusters** of consecutive disk blocks, and the clusters are linked. Clusters are sometimes called **file segments** or **extents**.

Indexed Allocation

- one or more index blocks contain pointers to the actual file blocks

File Headers

- A file header / file descriptor contains information about a file that is needed by the system programs that access the file records.
- The header includes
 - information that describes the file (field names and their data types) - record format descriptions
 - the addresses of the file blocks on disk

File Headers cont.

- To search for a record on disk, one or more blocks are copied into main memory buffers.
- Programs then search for the desired record or records within the buffers, using the information in the file header.
- If the address of the block that contains the desired record is not known, the search programs must do a linear search through the file blocks. (can be very time-consuming for a large file)
- **The goal of a good file organization is to locate the block that contains a desired record with a minimal number of block transfers.**

Operations on Files

- Mainly grouped into 2 types:
 - Retrieval operations
 - Update operations
- DBMS software programs, access records by using the following commands/operations;
 - Open, Reset, Find (Locate), Read (Get), FindNext, Delete, Modify, Insert, Close, Scan, FindAll, Find (Locate)_n, FindOrdered, Reorganize

File Organization and Access Method

- File Organization
 - File Organization is the organization of the data of a file into records, blocks, and access structures
 - This includes the way records and blocks are placed on the storage medium and interlinked
- Access method
 - Provides a group of operations that can be applied to a file

Static Files and Dynamic Files

- Static Files
 - update operations are rarely performed
 - Dynamic Files
 - may change frequently
 - update operations are constantly applied to them
- A successful file organization should perform as efficiently as possible the operations we expect to apply frequently to the file

Heap Files (Files of Unordered Records)

- Simplest and most basic type of organization
- Called as heap files or pile files
- New records are inserted at the end of the file – insertion is very efficient
- To search for a record, a linear search through the file records is necessary - an expensive procedure
- Deleting records leaves unused space in the disk block

Sorted Files (Files of Ordered Records)

- File records are kept sorted by the values of an ordering field
- Records must be inserted in the correct order - expensive
 - It is common to keep a separate unordered overflow (or transaction) file for new records to improve insertion efficiency.
 - This is periodically merged with the main ordered file.
- A binary search can be used to search for a record on its ordering field value.
- Reading the records in order of the ordering key values - extremely efficient

Average Access Times for Basic File Organizations

Type of Organization	Access/Search Method	Average Blocks to Access a Specific Record
Heap (unordered)	Sequential scan (linear search)	$b/2$
Ordered	Sequential scan	$b/2$
Ordered	Binary search	$\log_2 b$

Example:

- Assume a file with 1024 blocks ($b = 1024$) and if each block read takes, 10 ms.

File Type	Search Method	Average Blocks Accessed	Approx. Time (10 ms per block)
Heap (unordered)	Sequential scan	512	5.12 s
Ordered	Sequential scan	512	5.12 s
Ordered	Binary search	10	0.10 s

Try this by yourself.... 😊

Activity 05

- Find the advantages of ordered files over heap files.

Disk Storage, File Structures and Hashing

(Part 2 – Hashing Techniques)

Hashing Techniques

- Provides very fast access to records under certain search conditions.
- File organizations with hashing is called **hash files**.
- The search condition must be an equality condition on a single field, called the **hash field (hash key)**.

Hashing Techniques cont.

- A hash function or randomizing function is applied to the hash field value of a record to find the address of the disk block where the record is stored.
- Hashing is also used as an internal search structure whenever a group of records is accessed exclusively by using the value of one field.

Hashing Techniques cont.

- Several types;
 - Static Hashing Techniques
 - Internal Hashing
 - External Hashing
 - Dynamic Hashing Techniques
 - Extendible Hashing
 - Linear Hashing
 - Dynamic Hashing

Internal Hashing

- Used for internal files
- Implemented as a **hash table** through the use of an array of records

- A common hash function

$$h(K) = K \bmod M$$

(where K is the hash field value and M is the size of the array)

	Name	Ssn	Job	Salary
0				
1				
2				
3				
	⋮			
M - 2				
M - 1				

- Hash function transforms the hash field value into an integer between 0 and M-1.

Internal Hashing cont.

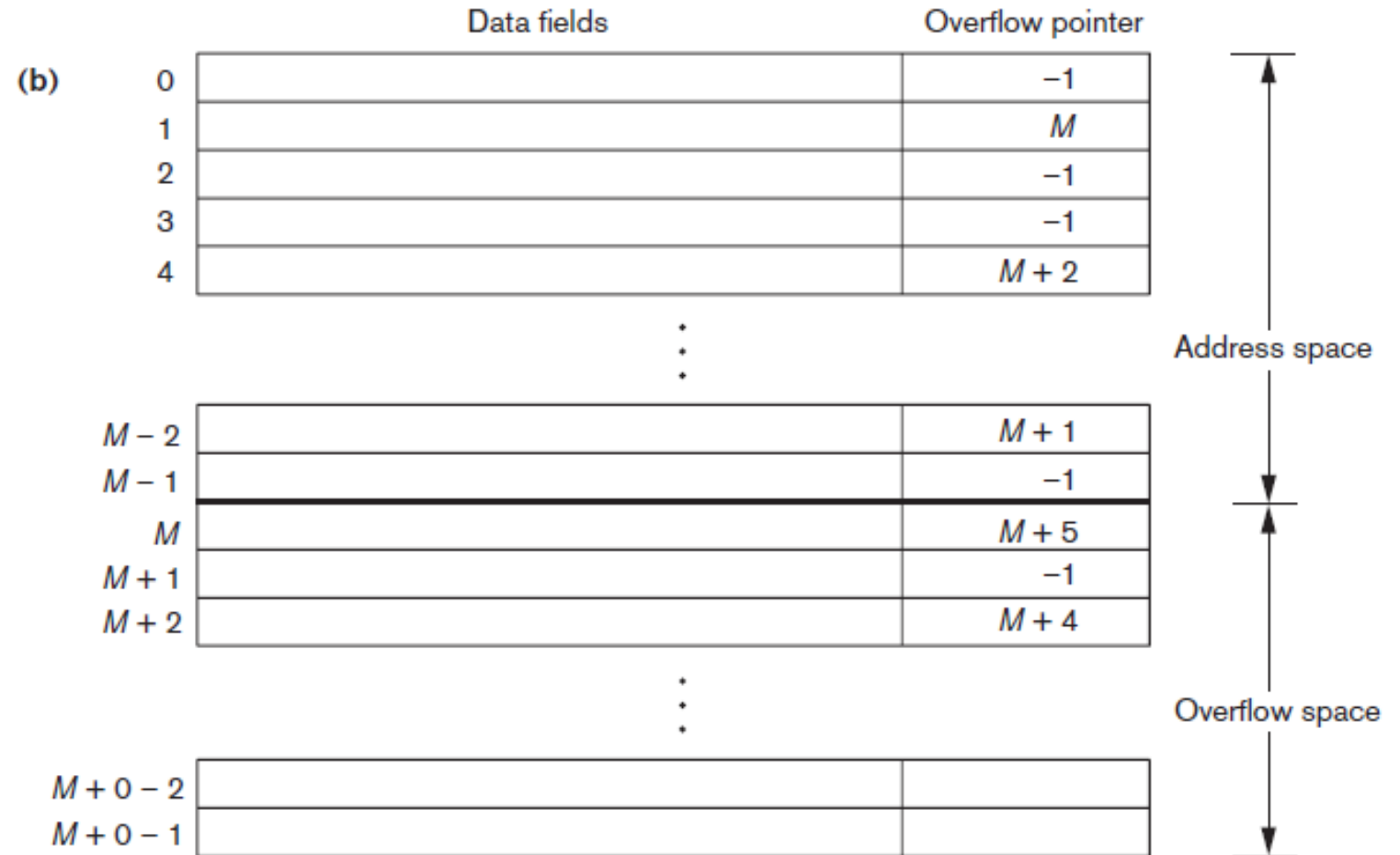
- A **collision** occurs when the hash field value of a record that is being inserted hashes to an address that already contains a different record.
- Have to find another position - **collision resolution**
- Collision resolution methods;
 - Open addressing
 - Chaining
 - Multiple hashing

Collision Resolution Methods

- **Open addressing:** Proceeding from the occupied position specified by the hash address, the program checks the subsequent positions in order until an unused (empty) position is found.
- **Chaining:** Various overflow locations are kept, usually by extending the array with a number of overflow positions. In addition, a pointer field is added to each record location. A collision is resolved by placing the new record in an unused overflow location and setting the pointer of the occupied hash address location to the address of that overflow location.
- **Multiple hashing:** The program applies a second hash function if the first results in a collision. If another collision results, the program uses open addressing or applies a third hash function and then uses open addressing if necessary.

Collision Resolution Methods cont.

- Collision resolution by chaining records



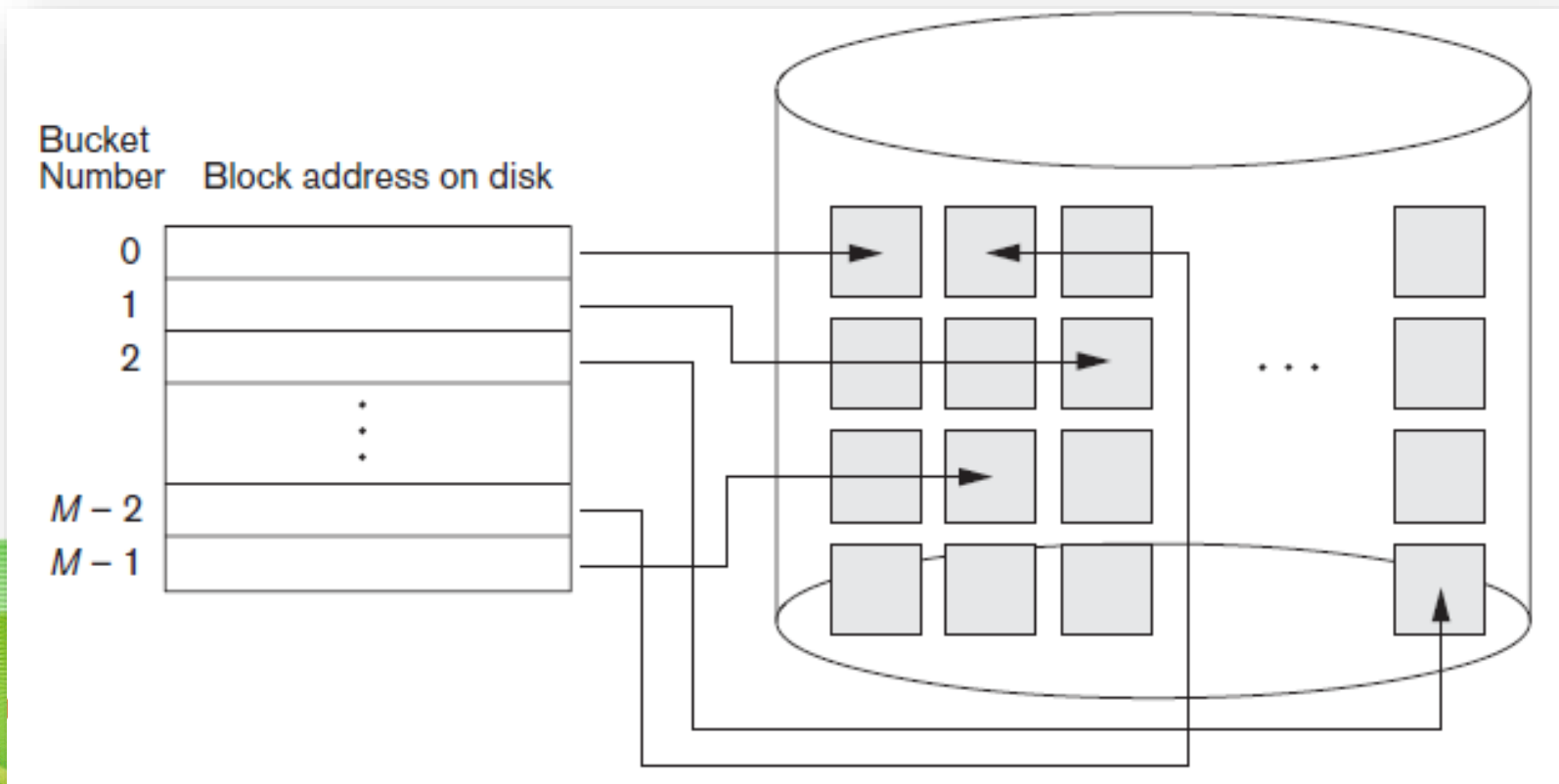
- null pointer = -1
- overflow pointer refers to position of next record in linked list

External Hashing

- Hashing for disk files is called **external hashing**.
- The target address space is made of buckets which hold multiple records.
- A bucket can be a disk block or a cluster of contiguous disk blocks.
- The hashing function maps a key into a relative bucket number.
- The collision problem is less severe with buckets, because as many records can fit into a bucket without causing problems.

External Hashing cont.

- A table maintained in the file header converts the bucket number into the corresponding disk block address.



External Hashing cont.

- Handling overflow for buckets by chaining

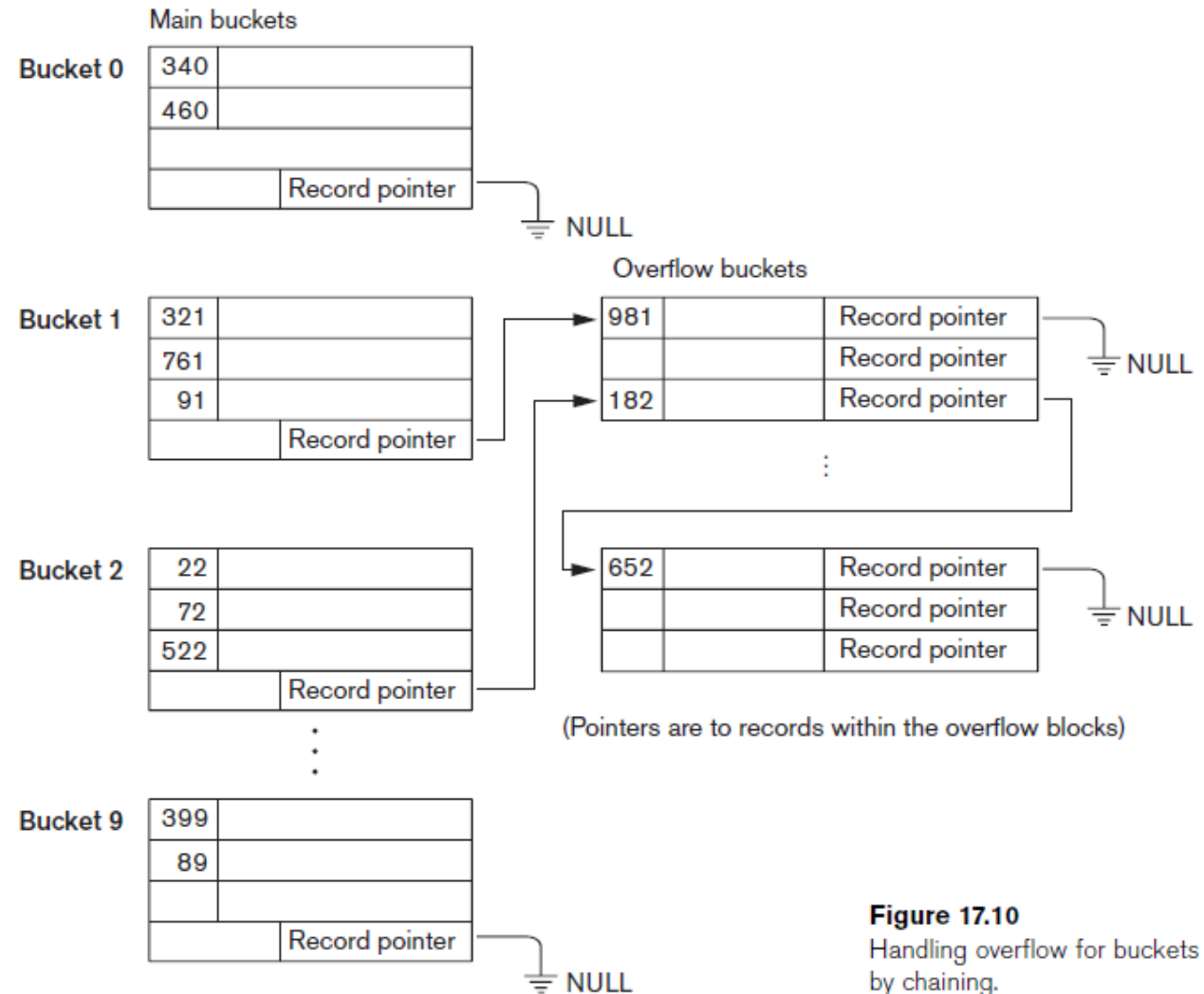


Figure 17.10
Handling overflow for buckets
by chaining.

- In static hashing,
 - fixed number of buckets M is allocated - a drawback for dynamic files
 - difficult to expand or shrink the file dynamically
- Assume that, an address space is having M buckets and number of records per bucket is m ;
 - then at most $(m * M)$ records will fit in the allocated space
 - If the number of records is fewer than $(m * M)$ leads to unused space
 - if the number of records increases than $(m * M)$, collisions will happen and retrieval will be slowed down

Extendible Hashing

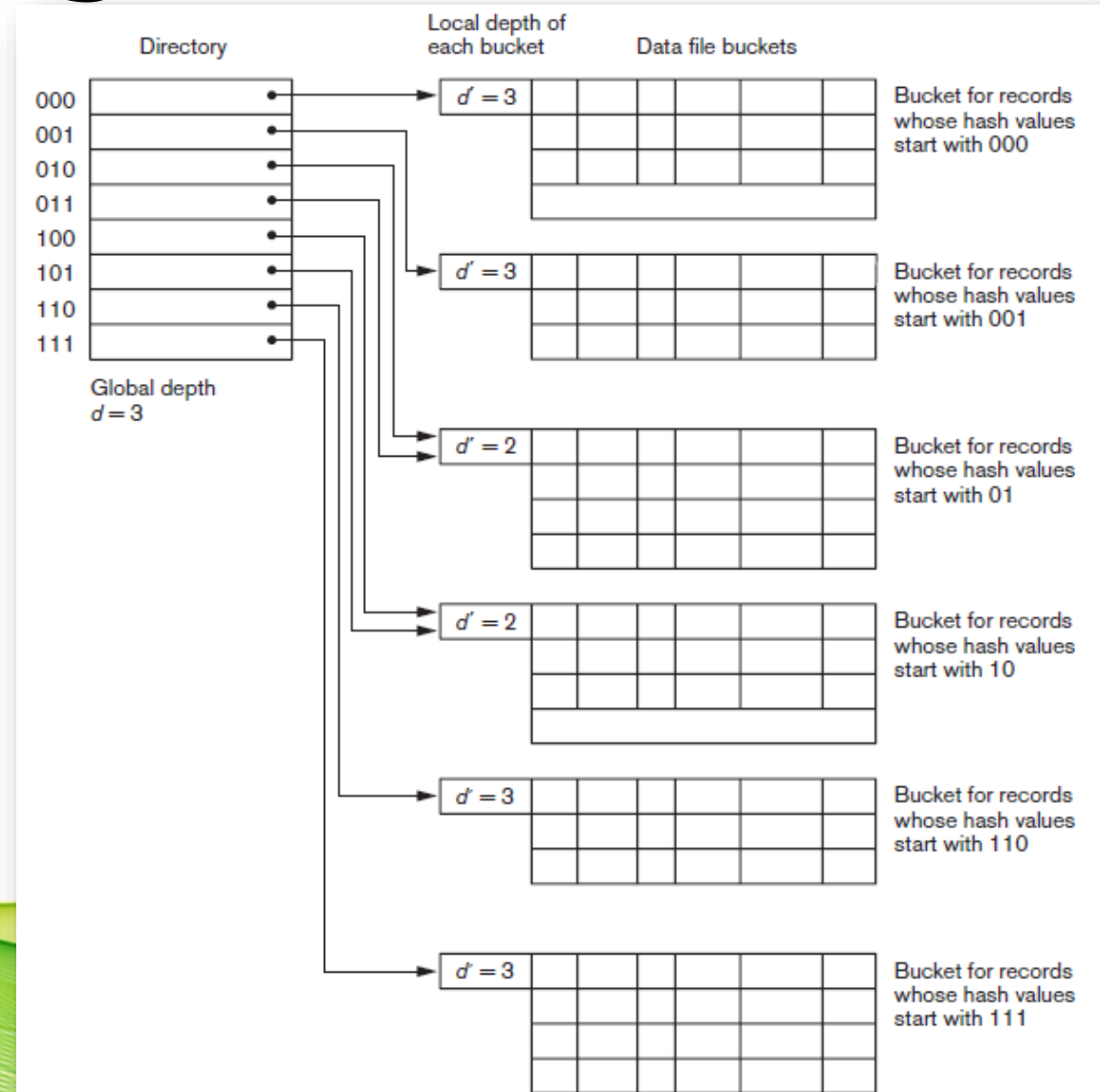
- Stores an access structure in addition to the file - somewhat similar to indexing
 - The main difference is;
 - In extendible hashing, the access structure is based on the values that result after applying the hash function to the search field.
 - In indexing, the access structure is based on the values of the search field itself.

Extendible Hashing cont.

- A type of directory - an array of 2^d bucket addresses is maintained where d is **global depth** of the directory.
- A **local depth** d' - stored with each bucket - specifies the number of bits on which the bucket contents are based.

Extendible Hashing cont.

Structure of the extendible hashing scheme



Extendible Hashing cont.

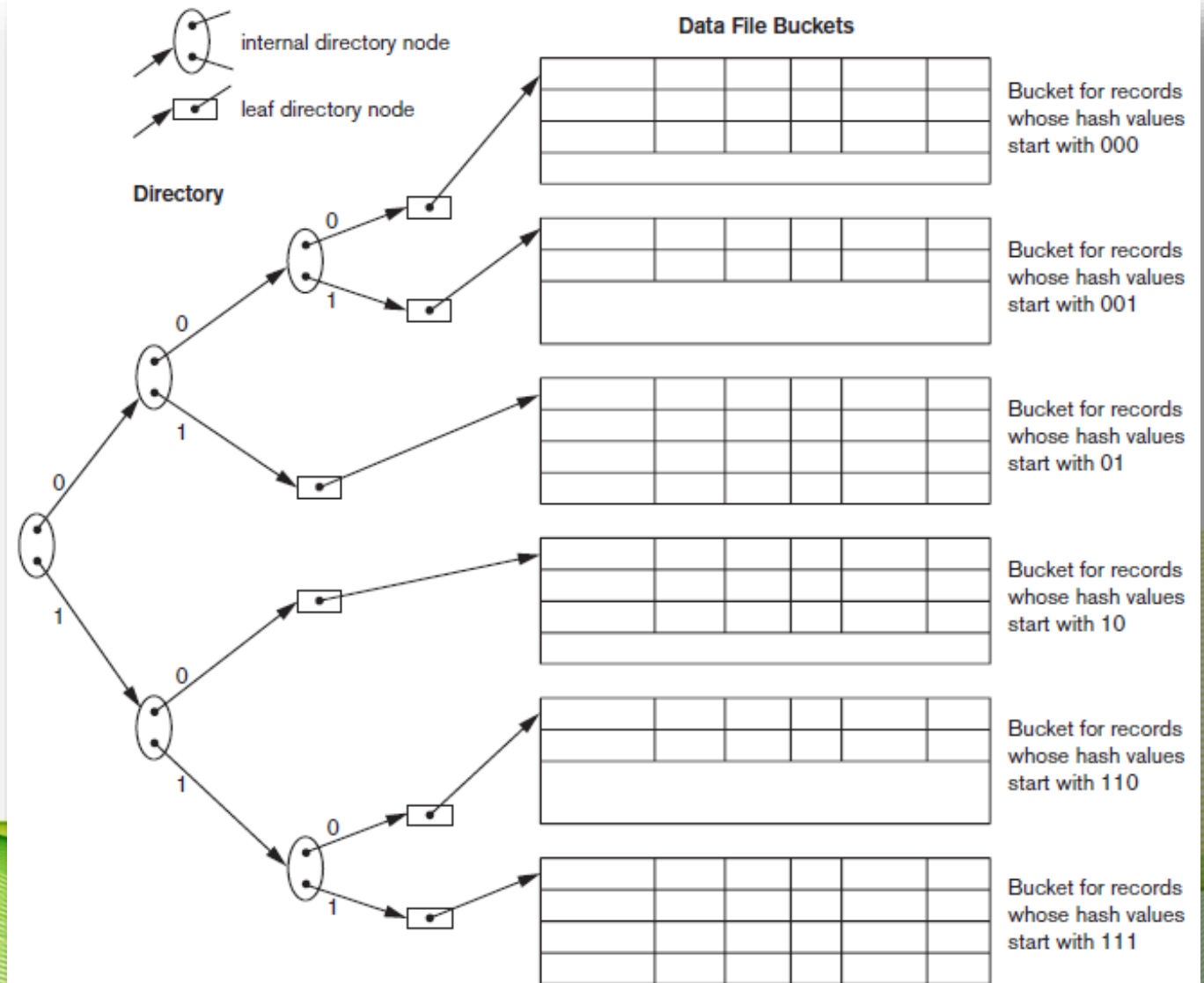
- Advantages:
 - The performance of the file does not degrade as the file grows
 - No space is allocated for future growth, but additional buckets can be allocated dynamically as needed
 - Splitting causes minor reorganization in most cases
- Disadvantages
 - The directory must be searched before accessing the buckets themselves - resulting in two block accesses instead of one in static hashing

Dynamic Hashing

- A precursor to extendible hashing was dynamic hashing
- Precursor - uses an access structure based on binary tree data structures
- Maintains a tree-structured directory with two types of nodes
 - Internal nodes that have two pointers
 - Left pointer corresponding to the 0 bit and a right pointer corresponding to the 1 bit.
 - Leaf nodes
 - Hold a pointer to actual bucket

Dynamic Hashing cont.

Structure of the dynamic hashing scheme



Linear Hashing

- Allow a hash file to expand and shrink its number of buckets dynamically without needing a directory.

Activity 06

- What is the difference between the directories of extendible hashing and dynamic hashing?

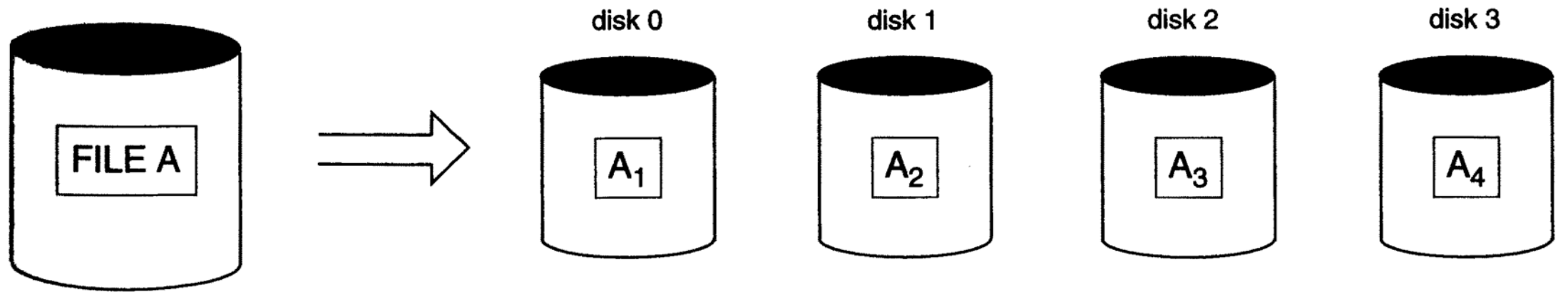
Parallelizing Disk Access Using RAID Technology

- Secondary storage technology must take steps to keep up in performance and reliability with processor technology.
- A major advance in secondary storage technology is represented by the development of **RAID**, which originally stood for **Redundant Arrays of Inexpensive Disks**.

Parallelizing Disk Access Using RAID Technology cont.

- RAID is a way to improve the performance and reliability of data storage.
- In this method, it combines multiple disks into a single logical unit, enabling simultaneous (parallel) data reading and writing across multiple disks. – this concept called **data striping** and utilizes **parallelism** to improve disk performance.
- Data striping distributes data transparently over multiple disks to make them appear as a single disk.
- This increases read/write speeds, enhances fault tolerance, and provides redundancy for data protection.

Parallelizing Disk Access Using RAID Technology cont.



Improving Reliability with RAID via Redundancy

- Store extra information that can be used to rebuild information lost in a disk failure
 - E.g., Mirroring(or shadowing)
 - Duplicate every disk. Logical disk consists of two physical disks.
 - Every write is carried out on both disks - Reads can take place from either disk.
 - If one disk in a pair fails, data still available in the other
 - Data loss will occur only if a disk fails, and its mirror disk also fails before the system is repaired
 - Probability of combined event is very small

Improving Performance with RAID via Parallelism

- Two main goals of parallelism in a disk system:
 - Load balance in multiple small accesses to increase throughput
 - Parallelize large accesses to reduce response time.
- Improve transfer rate by striping data across multiple disks.
 - Bit-level striping - split the bits of each byte across multiple disks
 - But seek/access time worse than for a single disk
 - Bit level striping is not used much any more
 - Block-level striping
 - Requests for different blocks can run in parallel if the blocks reside on different disks.
 - A request for a long sequence of blocks can utilize all disks in parallel.

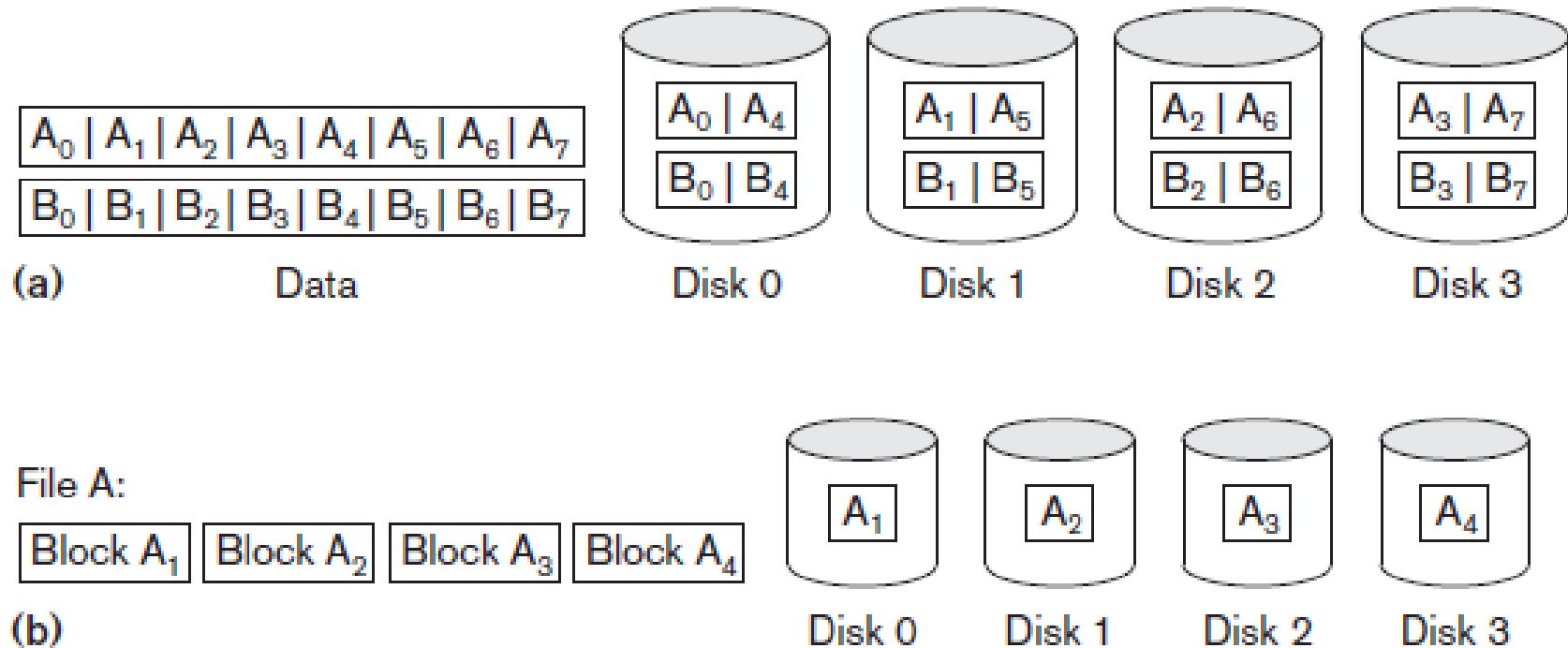
Improving Performance with RAID via Parallelism cont.

Figure 17.13

Striping of data across multiple disks.

(a) Bit-level striping across four disks.

(b) Block-level striping across four disks.



RAID Levels

- Different RAID organizations (**RAID levels**) were defined based on different combinations of the two factors of **granularity of data interleaving (striping)** and **pattern used to compute redundant information**.

RAID Levels cont.

- RAID level 0 has no redundant data and hence has the best write performance.
- RAID level 1 uses mirrored disks.
- RAID level 2 uses memory-style redundancy by using Hamming codes, which contain parity bits for distinct overlapping subsets of components. Level 2 includes both error detection and correction.
- RAID level 3 uses a single parity disk relying on the disk controller to figure out which disk has failed.
- RAID Levels 4 and 5 use block-level data striping, with level 5 distributing data and parity information across all disks.
- RAID level 6 applies the so-called $P + Q$ redundancy scheme using Reed-Soloman codes to protect against up to two disk failures by using just two redundant disks.